WAZUH SOC ANALYST HOME LAB — 14-DAY INTENSIVE PROGRAM

Process Chain Triage Lab

Context before verdict: reading process telemetry to decide whether a chain is malicious, and turning the finding into a Wazuh detection

Author	Aigbokhaode Hope Imomoh (Hope), SOC Analyst Trainee, Abuja, Nigeria
Report date	7 November 2026 | Version 1.0
Investigation date	7 November 2026 (investigating activity from 5 November 2026; screenshots show local time, UTC+1)
Platform	VirtualBox lab: Windows 10 VM (investigated host) and Ubuntu VM (Wazuh + Splunk)
Builds on	The PowerShell Detection Lab (same test activity, now investigated as an analyst would)
Evidence status	Every finding is tied to a screenshot taken during the investigation. Gaps are marked as gaps.

How to read this report. Non-technical readers: read the one-page summary and the boxes marked **IN PLAIN ENGLISH**. Beginners who want to repeat the investigation: go to Chapter 15. Reviewers who want proof: Chapter 9 and Appendix A.

The whole investigation in one page (no technical knowledge needed)

The question. Security tools raise alarms. A good analyst does not simply believe the alarm: they ask whether it is really a problem. This lab practises that question on the suspicious-looking PowerShell activity from the previous lab, using only the computer's own records of which programs started which.

The analogy. Imagine a shop with a visitor log. Every person who enters is recorded with *who let them in*, *what they said*, *who they are* and *what they touched*. Someone walking in wearing a mask and whispering is suspicious. But if the manager brought them in, the whispering was a rehearsal, and they touched nothing and left nothing behind, the picture changes. That is investigating **in context**.

- **Who let them in?** (parent process) — Two PowerShell windows, opened from the signed-in desktop session of user HR-STAFF1 but running with administrator rights.
- **What did they say?** (command line) — Scrambled (encoded) commands, a safety-setting bypass, and a hidden window.
- **What did they do next?** (child processes) — Only displayed the user name and computer name.
- **What did they touch?** (network and files) — No internet connections, no DNS lookups, no downloads. Only a tiny test file that PowerShell writes every time it starts.
- **Did the recorder miss anything?** — We checked: Sysmon dropped no events in the time window, and no "process injection" alerts occurred then.

> **VERIFIED RESULT**
> Conclusion: the activity **behaves like** a classic attacker technique, but in context it is an **authorised test** by the lab owner. Nothing in the evidence shows harm: no network traffic, no payload, no persistence found in the checks performed. The behaviour is a true detection, the intent is benign.

Why it matters. Labelling something "malicious" without context wastes time and can cause real damage (locking out a colleague, wiping a machine). Labelling it "fine" without context can let an attacker through. This report shows how to look at both sides before deciding.

Table of Contents

The whole investigation in one page (no technical knowledge needed) 2

Table of Contents 3

List of Figures 6

List of Tables 6

Chapter 1: Executive Summary 8

1.1 Purpose . . . 8

1.2 Method . . . 8

1.3 Key findings . . . 8

1.4 Not proven . . . 8

Chapter 2: Introduction and Objectives 10

2.1 The brief . . . 10

2.2 Scope . . . 10

2.3 Objectives mapped to outcomes . . . 10

Chapter 3: Technical Background 12

3.1 Key ideas . . . 12

3.2 Why context matters . . . 12

3.3 About the PSScriptPolicyTest file . . . 12

Chapter 4: Lab Environment and Architecture 13

Chapter 5: Prerequisites and Safety 14

Chapter 6: Step-by-Step Investigation 15

Step 1 – Check what Sysmon records . . . 15

Step 2 – Build the process tree . . . 15

Step 3 – Trace the ancestors . . . 17

Step 4 – Search for network and file activity . . . 18

Step 5 – Calibrate what “normal” looks like in Splunk . . . 19

Step 6 – Rule out injection and Sysmon blind spots . . . 21

Step 7 – Turn the finding into a Wazuh detection . . . 23

Step 8 – Record the conclusion . . . 27

Chapter 7: Investigation Logic and Why Context Matters 28

7.1 True positive, benign intent . . . 28
7.2 What would change the verdict . . . 28
7.3 The new detection: rule 100610 . . . 28

Chapter 8: Results and Conclusion 30

8.1 Test results . . . 30
8.2 Timeline . . . 30
8.3 Investigation notes . . . 30
8.4 The six analyst questions . . . 31
8.5 Analyst conclusion . . . 31

Chapter 9: Evidence and Screenshot Analysis 32

Chapter 10: Troubleshooting and Root-Cause Analysis 33

Chapter 11: Security Findings and Risk Analysis 34

Chapter 12: Mitigation and Remediation 35

Chapter 13: What I Learned 36

Chapter 14: Practical Knowledge and Competencies 37

Chapter 15: Reproduction Guide (for a beginner) 38

15.1 What you need . . . 38
15.2 Create something to investigate (only if you have no earlier test) . . . 38
15.3 Run the six checks . . . 38
15.3b Optional: turn it into a Wazuh detection . . . 38
15.4 Write the conclusion . . . 38
15.5 Troubleshooting . . . 38

Chapter 16: Limitations and Future Improvements 40

Future improvements . . . 40

Chapter 17: Conclusion 41

Appendix A: Evidence Register 42

Appendix B: Commands Used 43

Appendix C: Splunk Query 44

Appendix D: Wazuh Rule 100610 (copy-ready) 45

Appendix E: Glossary in Plain Language 46

Appendix F: Integrity and Redaction Note 47

List of Figures

Figure 1 – The investigation brief (source of tasks, deliverables and analyst questions) 10

Figure 2 – Sysmon event counts by type (latest 5,000 events) 15

Figure 3 – Every process created between 3:05 and 3:08 PM on 5 Nov, with PID, parent PID, user and command line 16

Figure 4 – Process tree of the investigated chain, reconstructed from Sysmon Event 1 (times are local, 5 Nov 2026) 17

Figure 5 – PID 7672 was started by RuntimeBroker.exe (PID 6800) at 13:01:38 17

Figure 6 – From the earlier lab: the same RuntimeBroker PID 6800 shown with ParentUser HOPELABS\HR-STAFF1 (starting PowerShell PID 4568) 18

Figure 7 – Network, DNS and file events from the chain's processes: only five __PSScriptPolicyTest files 19

Figure 8 – Benign comparison: 32-bit PowerShell as SYSTEM writes the same kind of file to C:\Windows\SystemTemp (7 Nov, 11:21 AM) 20

Figure 9 – Benign comparison: Administrator's PowerShell writes the same file to its Temp folder (7 Nov, 11:19 AM) 20

Figure 10 – Benign comparison: Edge WebView2 DNS lookup as HR-STAFF1 (7 Nov, 11:39 AM) 21

Figure 11 – Splunk event from inside the investigated window: svchost.exe checks internet connectivity (5 Nov, 3:07:43 PM; photo of screen) 21

Figure 12 – Event 8 and 255 entries since 3 Nov: none between 3:00 and 3:10 PM on 5 Nov 22

Figure 13 – Event 8 entries: WerFault.exe into sihost.exe (28 Oct) and an unknown process into PowerShell (3 Nov) 22

Figure 14 – Event 8 on 21 Oct: unknown process into cmd.exe with CtrlRoutine; more Event 255 entries 23

Figure 15 – Wazuh Threat Hunting: four alerts for rule 100610 (13:40:19 and 13:45:03 on 7 Nov) 24

Figure 16 – Rule 100610 alert metadata: level 12 and MITRE T1059.001, T1033, T1082 25

Figure 17 – Alert for whoami.exe: the discovery command seen by Wazuh 25

Figure 18 – Alert for whoami.exe: its parent is the encoded PowerShell 26

Figure 19 – Alert for HOSTNAME.EXE: the same encoded parent (second run) 26

Figure 20 – The failed test: pasted lines merged into encenc, giving "not properly encoded" (7 Nov, 1:40 PM) 33

List of Tables

Table 1 – Headline findings 8

Table 2 – Lab tasks and deliverables 10

Table 3 – Concepts used in the investigation 12

Table 4 – Machines and tools used 13

Table 5 – What the command does 16

Table 6 – Table 5 – Process tree data (from the figure above) 16

Table 7 – Indicators weighed in the investigation 28

Table 8 – Rule 100610 line by line 28

Table 9 – Checks, results and status 30

Table 10 – Key times (5 Nov, local) 30

Table 11 – Investigation notes 30

Table 12 – Answers tied to evidence 31

Table 13 – Deliverables and their evidence 32

Table 14 – Issues, causes and fixes 33
Table 15 – Findings 34
Table 16 – Implemented versus recommended 35
Table 17 – Skills mapped to evidence 37
Table 18 – Quick guide 39
Table 19 – Figure-to-original evidence register 42
Table 20 – Glossary 46

Chapter 1: Executive Summary

> **IN PLAIN ENGLISH**
> We took the PowerShell commands from the previous lab and investigated them like a SOC analyst: who started them, what they ran, what they touched, and whether the monitoring itself could be trusted. Verdict: suspicious technique, benign intent.

1.1 Purpose

The brief asks the analyst to *"use process telemetry to determine whether a process chain deserves investigation"* and to document *why context matters before labelling activity malicious*. Required deliverables: a process tree, investigation notes, evidence screenshots and an analyst conclusion.

1.2 Method

1. Check which Sysmon event types are recorded (process, network, file).
2. Reconstruct the process tree for 5 Nov, 3:05–3:08 PM.
3. Trace the tree upward to find what started the PowerShell consoles.
4. Search for network, DNS and file activity from every process in the chain.
5. Compare with ordinary Windows activity to calibrate what "normal" looks like.
6. Rule out process injection and Sysmon blind spots.
7. Turn the finding into a Wazuh rule (100610) and confirm it fires; write the conclusion.

1.3 Key findings

Table 1 – Headline findings

Question	Finding	Evidence
Process chain	Two PowerShell consoles (PIDs 7672, 7240) each launched an encoded and a hidden PowerShell process; the encoded ones launched whoami.exe and HOSTNAME.EXE	Fig. 3, Fig. 4
What started it	Both consoles were started by RuntimeBroker.exe (PID 6800), which belongs to the signed-in desktop user HR-STAFF1; the consoles ran as HOPELABS\Administrator	Fig. 5, Fig. 6
Network activity	None: no Event 3 and no DNS (Event 22) from any process in the chain	Fig. 7
File activity	Five files, all __PSScriptPolicyTest_*.ps1 in the Administrator Temp folder, one per PowerShell start: normal PowerShell behaviour	Fig. 7, Fig. 8, Fig. 9
Injection check	Three Event 8 entries exist in Sysmon since mid-October; none fall in the window; their patterns look routine	Fig. 13, Fig. 14, Fig. 12
Monitoring gaps	Sysmon dropped events on 5 Nov at 12:58 PM, over two hours before the test; none in the test window	Fig. 12
Wazuh detection	New rule 100610 (level 12; MITRE T1059.001, T1033, T1082) fired four times across two test runs, once for whoami.exe and once for HOSTNAME.EXE per run	Fig. 15, Fig. 16
Verdict	Suspicious technique, benign intent: authorised lab test	Chapter 8

1.4 Not proven

- Registry changes (Sysmon Event 13) and scheduled tasks were not examined, so "no persistence" is limited to what was checked.

- The false-positive rate of the new rule 100610 was not measured, and the rule syntax-check and restart output were not captured.
- The Splunk summary table (count by process) was not captured; Splunk evidence is individual events.
- That the two consent.exe entries are the elevation prompts for these consoles is likely but not proven.

Chapter 2: Introduction and Objectives

2.1 The brief

The brief below is the source of the tasks, deliverables and analyst questions, and the layout reference for this report.



DAY 7 — SUSPICIOUS PROCESS INVESTIGATION

Objective: Use process telemetry to determine whether a process chain deserves investigation.

Lab Tasks

- Review process creation telemetry.
- Identify parent and child processes.
- Review command-line context.
- Identify the user and endpoint.
- Search for surrounding network/file activity.
- Document why context matters before labeling activity malicious.

Required Deliverables

- Process tree
- Investigation notes
- Evidence screenshots
- Analyst conclusion

Analyst Questions

- What happened?
- How was it detected?
- What evidence supports the finding?
- Which host and account were involved?

Wazuh SOC Analyst Home Lab — 14-Day Intensive Program Page 9

- What other systems or events should be checked?
- What action would you take in a real SOC?

Figure 1 – The investigation brief (source of tasks, deliverables and analyst questions)

2.2 Scope

In scope: the PowerShell chain from 5 Nov 3:05–3:08 PM on DESKTOP-T6T1N2V, investigated with Sysmon, Splunk and PowerShell queries. **Out of scope:** the Kali and domain-controller VMs, malware analysis, and any production system.

2.3 Objectives mapped to outcomes

Table 2 – Lab tasks and deliverables

#	Requirement	Outcome
1	Review process creation telemetry	Done: Fig. 2, Fig. 3
2	Identify parent and child processes	Done: Fig. 4
3	Review command-line context	Done: Chapter 8 notes
4	Identify the user and endpoint	Done: Administrator; DESKTOP-T6T1N2V; parent session HR-STAFF1
5	Search surrounding network/file activity	Done: Fig. 7; Splunk comparison
6	Document why context matters	Done: Chapter 7 and Section 8.5
E1	Extra: detect the chain in Wazuh	Done: rule 100610 (Fig. 15, Fig. 16)
D1	Process tree	Fig. 4, Table 5
D2	Investigation notes	Section 8.3

#	Requirement	Outcome
D3	Evidence screenshots	Chapter 9, Appendix A
D4	Analyst conclusion	Section 8.5

Chapter 3: Technical Background

> **IN PLAIN ENGLISH**
> A “process” is simply a running program. Windows records which program started which, like a family tree. Learning to read that tree is the core skill of this lab.

3.1 Key ideas

Table 3 – Concepts used in the investigation

Term	Plain meaning	Why it matters
Process	A running program	Everything on a computer happens inside processes
Parent / child	The program that started another program / the one that was started	Attacker chains look unusual (for example a document starting PowerShell)
Process tree	The parent-child family tree	Shows how an activity unfolded
Command line	The exact text used to start a program	Reveals flags like -EncodedCommand
PID / ProcessGuid	Number / unique ID for a process	PIDs get reused; the GUID is unique
Integrity level	How much power the process has (Medium, High, System)	High means administrator rights
LOLBin	“Living off the land” binary: a normal Windows tool misused	whoami and hostname are normal but are used for reconnaissance
Telemetry	Recorded activity data	The evidence we investigate
Sysmon Events	1 process created, 3 network connection, 11 file created, 22 DNS query, 8 remote thread, 255 Sysmon error	Each answers a different question

3.2 Why context matters

The same behaviour can be good or bad depending on **who** did it, **from where**, **why**, and **what happened next**. A *true positive* means the detection correctly spotted the behaviour; whether the behaviour is *malicious* is a separate decision made with context. The investigation weighs indicators on both sides rather than reacting to one scary-looking flag.

3.3 About the PSScriptPolicyTest file

When PowerShell starts, it writes a tiny file named `__PSScriptPolicyTest_<random>.ps1` to a temporary folder to test whether script execution is restricted on that system. It looks odd, which is exactly why it is a good teaching example: **an unfamiliar-looking name is not evidence of malice** without context.

Chapter 4: Lab Environment and Architecture

Table 4 – Machines and tools used

Component	Details (from the evidence)
Windows 10 VM	DESKTOP-T6T1N2V (domain HopeLabs.Local), 192.168.56.109, Wazuh agent ID 006, Sysmon installed; users HOPELABS\Administrator (tests) and HOPELABS\HR-STAFF1 (signed-in desktop user)
Ubuntu VM	192.168.56.101; Wazuh manager and dashboard; Splunk Enterprise 9.4.2 (indexes sysmon, windows, main)
Telemetry used	Sysmon Events 1, 3, 11, 22, 8 and 255, read locally with PowerShell and in Splunk
Time	Investigation on 7 Nov 2026; activity of 5 Nov 2026 3:05–3:08 PM local (UTC+1)

The architecture is the same as in the PowerShell Detection Lab: Windows records, the agent collects, the manager and Splunk store. This lab mainly reads the **Sysmon** record, locally and in Splunk.

Chapter 5: Prerequisites and Safety

- Completed PowerShell Detection Lab (or any Windows host with Sysmon recording process, network and file events).
- Administrator PowerShell on Windows; Splunk (optional) on the Ubuntu VM with the Sysmon data indexed.
- Know the **time window** to investigate and the local time zone (here UTC+1).
- Take a VM snapshot before you begin.

> **CAUTION**
> This lab only *reads* logs. Do not delete or edit Sysmon logs while investigating: evidence must stay intact. Do not run unknown commands you find in events.

> **NOTE**
> Dates move on. On 6 Nov, “Last 24 hours” in Splunk still reached 5 Nov; on 7 Nov it did not. Always state the exact window in the query (see Chapter 10).

Chapter 6: Step-by-Step Investigation

> **IN PLAIN ENGLISH**
> Eight steps. Each one asks one question of the evidence; step 7 turns what you learned into a Wazuh detection.

Step 1 – Check what Sysmon records

Where	Windows 10, PowerShell as Administrator (7 Nov, about 11:20 AM)
Why	You cannot investigate network or file activity if Sysmon is not recording it.
Expected	Event IDs 1, 3 and 11 appear.
Actually observed	Event 1 (2,295), 13 (1,599), 11 (649), 22 (260), 3 (94) and smaller counts of 12, 5, 2, 4, 255 and 8 across the latest 5,000 events (about 3–7 Nov).

Windows 10, elevated PowerShell

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5000 |
  Group-Object Id | Sort-Object Count -Descending |
  Format-Table Count, Name -AutoSize
```

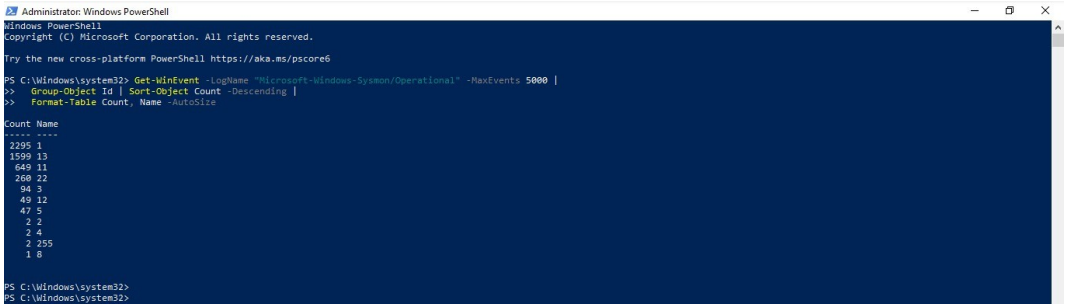


Figure 2 – Sysmon event counts by type (latest 5,000 events)

Shows	Counts per Event ID: 1 (2295), 13 (1599), 11 (649), 22 (260), 3 (94), 12 (49), 5 (47), 2, 4 and 255 (2 each), 8 (1).
Demonstrates	Process, network, file and DNS events are all recorded, so absence of network/file events later is meaningful.
Limitation	Counts cover only the latest 5,000 events.

Step 2 – Build the process tree

Where	Windows 10, elevated PowerShell (7 Nov, about 11:27 AM)
Why	The tree shows how the activity unfolded and who is related to whom.
Expected	Rows for the test PowerShell processes, whoami.exe and HOSTNAME.EXE, mixed with background noise.
Actually observed	All expected processes found, with parent PIDs; Windows Update, UAC, Edge and Copilot processes appear alongside.

Windows 10, elevated PowerShell

```
$s = Get-Date "2026-11-05 15:05:00"; $e = Get-Date "2026-11-05 15:08:00"
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1; StartTime=$s; EndTime=$e} |
 ForEach-Object {
  $x = [xml]$_.ToXml(); $d = @{}
  $x.Event.EventData.Data | ForEach-Object { $d[$_.Name] = $_.'#text' }
  [pscustomobject]@{
   Time = $_.TimeCreated.ToString('HH:mm:ss.fff'); PID = $d.ProcessId; PPID = $d.ParentProcessId
   User = $d.User; Image = Split-Path $d.Image -Leaf; Cmd = $d.CommandLine }
 } | Sort-Object Time | Format-Table -AutoSize -Wrap
```

Table 5 – What the command does

Part	Meaning
StartTime / EndTime	Limits the search to the 3-minute window
Id=1	Only “process created” events
[xml]$_.ToXml()	Turns each event into labelled fields (ProcessId, ParentProcessId, User, CommandLine)
PID / PPID	Process ID and parent process ID: the links that form the tree
Sort-Object Time	Shows the order things happened

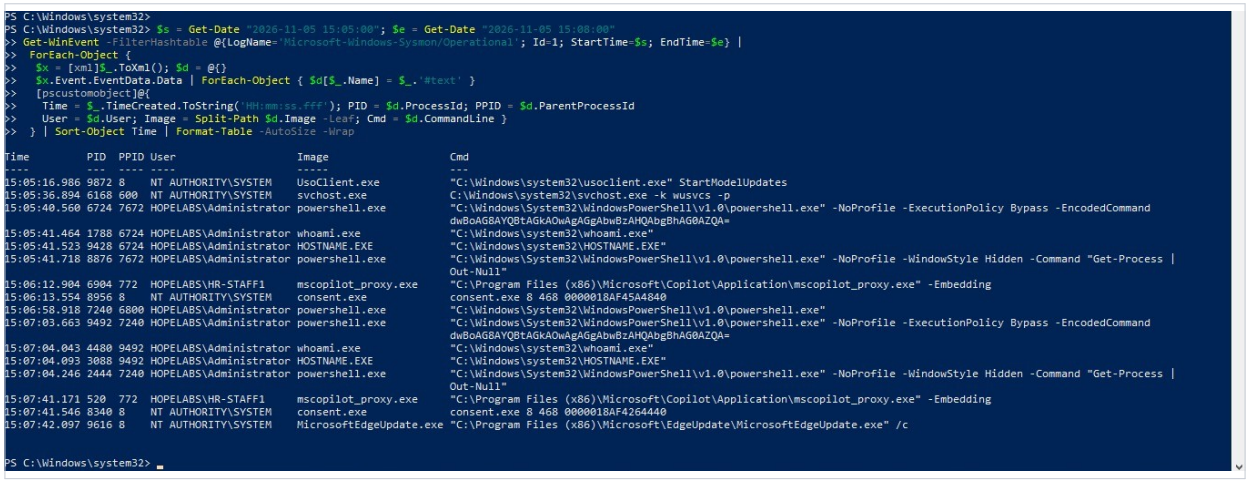


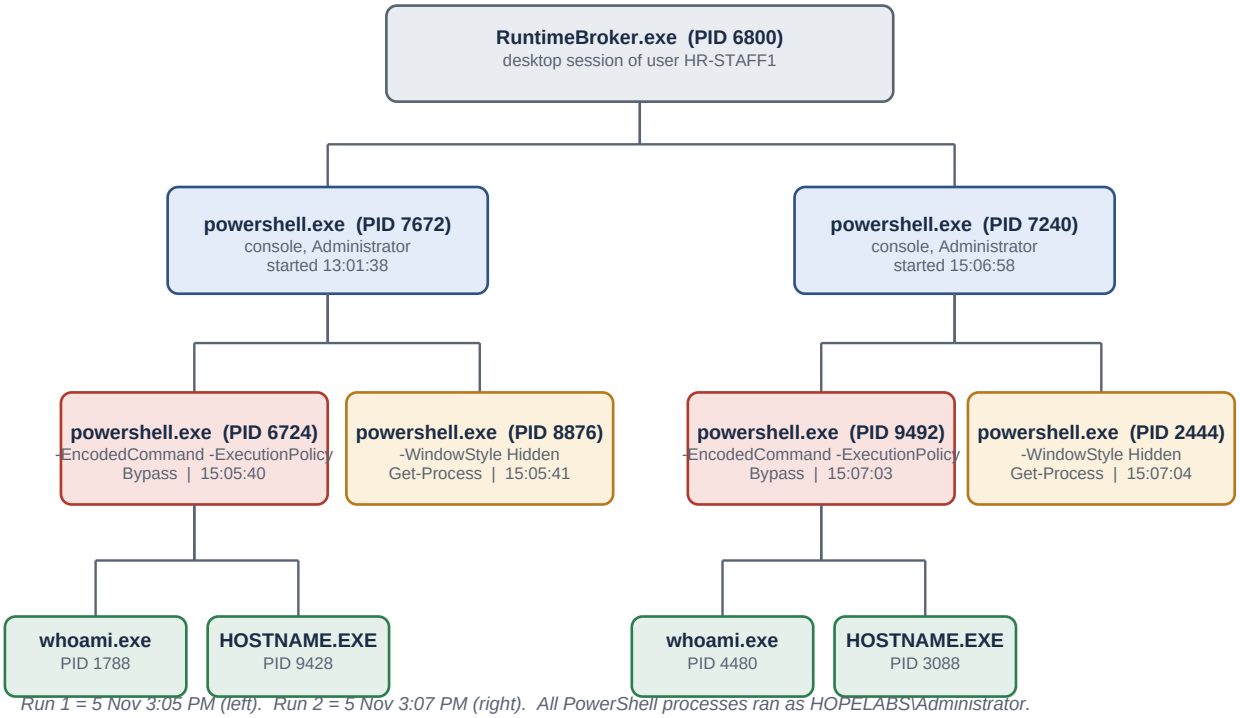
Figure 3 – Every process created between 3:05 and 3:08 PM on 5 Nov, with PID, parent PID, user and command line

Reading the table

Table 6 – Table 5 – Process tree data (from the figure above)

Time	PID	PPID	User	Process and key flags	In chain?
15:05:40	6724	7672	Administrator	powershell – -ExecutionPolicy Bypass -EncodedCommand dwBoAG8A...	Yes
15:05:41	1788 / 9428	6724	Administrator	whoami.exe / HOSTNAME.EXE	Yes
15:05:41	8876	7672	Administrator	powershell – -WindowStyle Hidden -Command Get-Process	Yes
15:06:58	7240	6800	Administrator	powershell (console for run 2)	Yes
15:07:03	9492	7240	Administrator	powershell – -EncodedCommand ...	Yes
15:07:04	4480 / 3088	9492	Administrator	whoami.exe / HOSTNAME.EXE	Yes
15:07:04	2444	7240	Administrator	powershell – -WindowStyle Hidden ...	Yes
15:05:16 / 15:05:36	9872 / 6168	8 / 600	SYSTEM	UsoClient.exe, svchost.exe -k wusvcs (Windows Update)	No
15:06:12 / 15:07:41	6904 / 520	772	HR-STAFF1	mscopilot_proxy.exe -Embedding (Edge/Copilot)	No

Time	PID	PPID	User	Process and key flags	In chain?
15:06:13 / 15:07:41	8956 / 8340	8	SYSTEM	consent.exe (permission prompt)	Unproven
15:07:42	9616	8	SYSTEM	MicrosoftEdgeUpdate.exe /c	No



Run 1 = 5 Nov 3:05 PM (left). Run 2 = 5 Nov 3:07 PM (right). All PowerShell processes ran as HOPELABS\Administrator.

Figure 4 – Process tree of the investigated chain, reconstructed from Sysmon Event 1 (times are local, 5 Nov 2026)

Step 3 – Trace the ancestors

Where	Windows 10, elevated PowerShell (7 Nov, about 11:32 AM)
Why	A tree is only as good as its top: who started the consoles?
Expected	The parent of PID 7672 and PID 6800.
Actually observed	PID 7672 was started at 13:01:38 on 5 Nov by RuntimeBroker.exe (PID 6800) as HOPELABS\Administrator. No Event 1 exists for PID 6800 in the 12:00–15:07 window.

Windows 10, elevated PowerShell

```
$s = Get-Date "2026-11-05 12:00:00"; $e = Get-Date "2026-11-05 15:07:00"
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1; StartTime=$s; EndTime=$e} |
 ForEach-Object {
  $x = [xml]$_.ToXml(); $d = @{}
  $x.Event.EventData.Data | ForEach-Object { $d[$_.Name] = $_.'#text' }
  if ($d.ProcessId -in 7672,6800) {
   [pscustomobject]@{ Time=$_.TimeCreated.ToString('HH:mm:ss'); PID=$d.ProcessId; PPID=$d.ParentProcessId
    User=$d.User; Image=Split-Path $d.Image -Leaf; Parent=Split-Path $d.ParentImage -Leaf; Cmd=$d.CommandLine } }
 } | Format-Table -AutoSize -Wrap
```

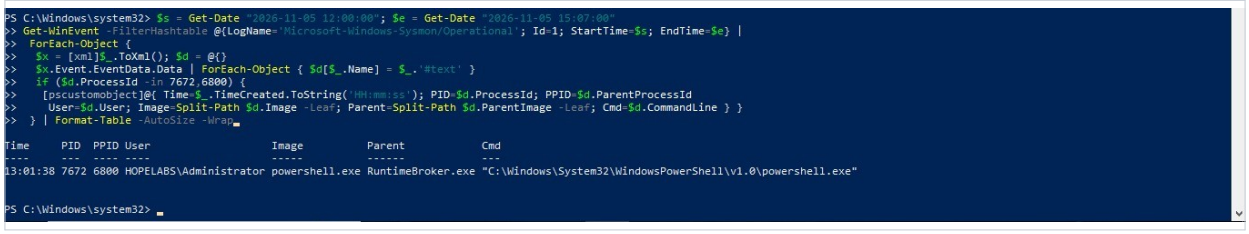


Figure 5 – PID 7672 was started by RuntimeBroker.exe (PID 6800) at 13:01:38

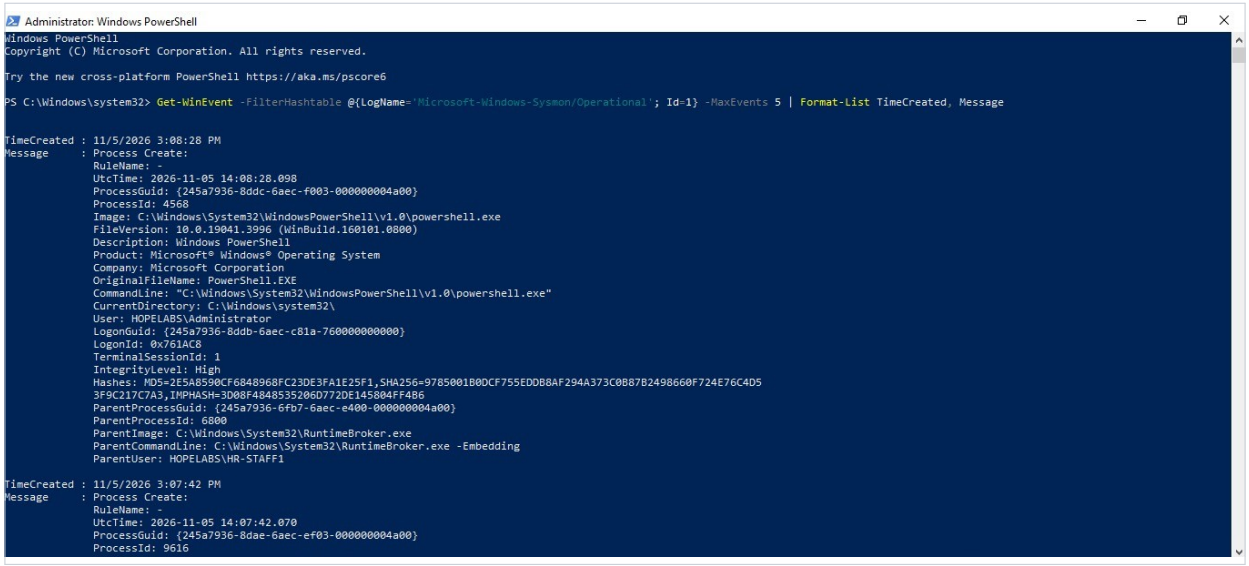


Figure 6 – From the earlier lab: the same RuntimeBroker PID 6800 shown with ParentUser HOPELABS\HR-STAFF1 (starting PowerShell PID 4568)

Shows	ParentProcessId 6800, ParentImage RuntimeBroker.exe, ParentUser HOPELABS\HR-STAFF1, for a PowerShell run as HOPELABS\Administrator.
Demonstrates	RuntimeBroker PID 6800 belongs to HR-STAFF1's desktop session. Both test consoles share this parent, so they were opened from that session and elevated to Administrator.
Limitation	The link to 7672 and 7240 rests on the PID 6800 match (PIDs are unique at a given moment); the owner of 6800 is shown in this separate event.

Step 4 – Search for network and file activity

Where	Windows 10, elevated PowerShell (7 Nov, about 11:35 AM)
Why	What a process touches is as informative as what started it: downloads and outbound connections are what separate a test from an attack.
Expected	Either related events or an empty result (also a valid finding).
Actually observed	No network (Event 3) or DNS (Event 22) events from any chain process. Five file-create events (Event 11), one per PowerShell process.

Windows 10, elevated PowerShell

```
$p = 7672,7240,6724,9492,8876,2444,1788,9428,4480,3088
$s = Get-Date "2026-11-05 15:05:00"; $e = Get-Date "2026-11-05 15:08:00"
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=3,11,22; StartTime=$s; EndTime=$e} |
 ForEach-Object {
  $x = [xml]$_.ToXml(); $d = @{}
  $x.Event.EventData.Data | ForEach-Object { $d[$_.Name] = $_.'#text' }
  if ([int]$d.ProcessId -in $p) {
   [pscustomobject]@{ Time=$_.TimeCreated.ToString('HH:mm:ss'); Evt=$_.Id; PID=$d.ProcessId
    Image=Split-Path $d.Image -Leaf
    Detail=($d.TargetFilename, $d.QueryName, ($d.DestinationIp+':'+$d.DestinationPort) | Where-Object { $_ -and $_ -ne ':' } | Select-Object -First 1) } }
 } | Sort-Object Time | Format-Table -AutoSize -Wrap
```

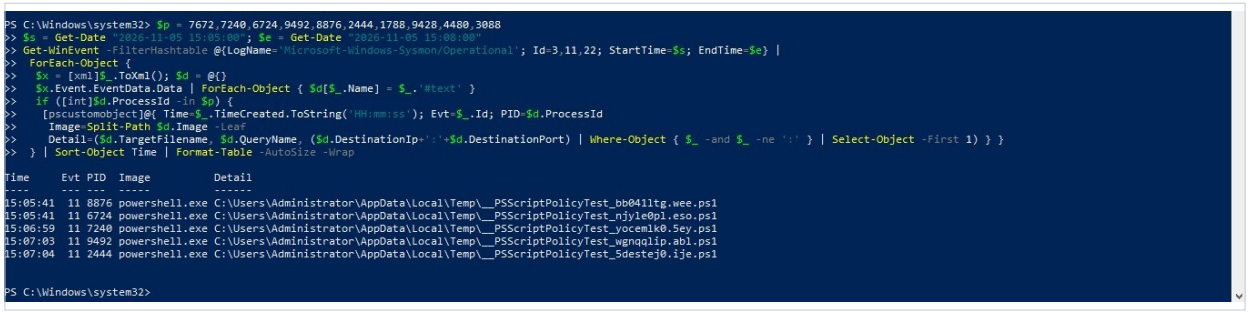


Figure 7 – Network, DNS and file events from the chain's processes: only five __PSScriptPolicyTest files

Shows	Five Event 11 rows: PIDs 8876, 6724, 7240, 9492 and 2444, each writing C:\Users\Administrator\AppData\Local\Temp__PSScriptPolicyTest_<random>.ps1 within about a second of starting. No Event 3 or 22 rows.
Demonstrates	The chain made no outbound connections or DNS lookups and dropped no payloads. whoami.exe and HOSTNAME.EXE wrote nothing.
Limitation	Covers 3:05–3:08 PM only; Sysmon logging gaps are checked in Step 6.

Step 5 – Calibrate what "normal" looks like in Splunk

Where	Ubuntu VM, Splunk Search (7 Nov, 11:47 AM–12:02 PM)
Why	To judge whether the chain is unusual, see what ordinary activity writes and connects to.
Expected	Routine activity from other processes and users.
Actually observed	The same __PSScriptPolicyTest file is written by PowerShell under other accounts (SYSTEM, Administrator) and folders; Edge, spoolsv, lsass and svchost make routine DNS lookups.

Splunk Search; state the window in the query

```
index=sysmon (EventCode=3 OR EventCode=11 OR EventCode=22)
  earliest="11/05/2026:15:05:00" latest="11/05/2026:15:08:00"
  | stats count by EventCode, Image, User
```

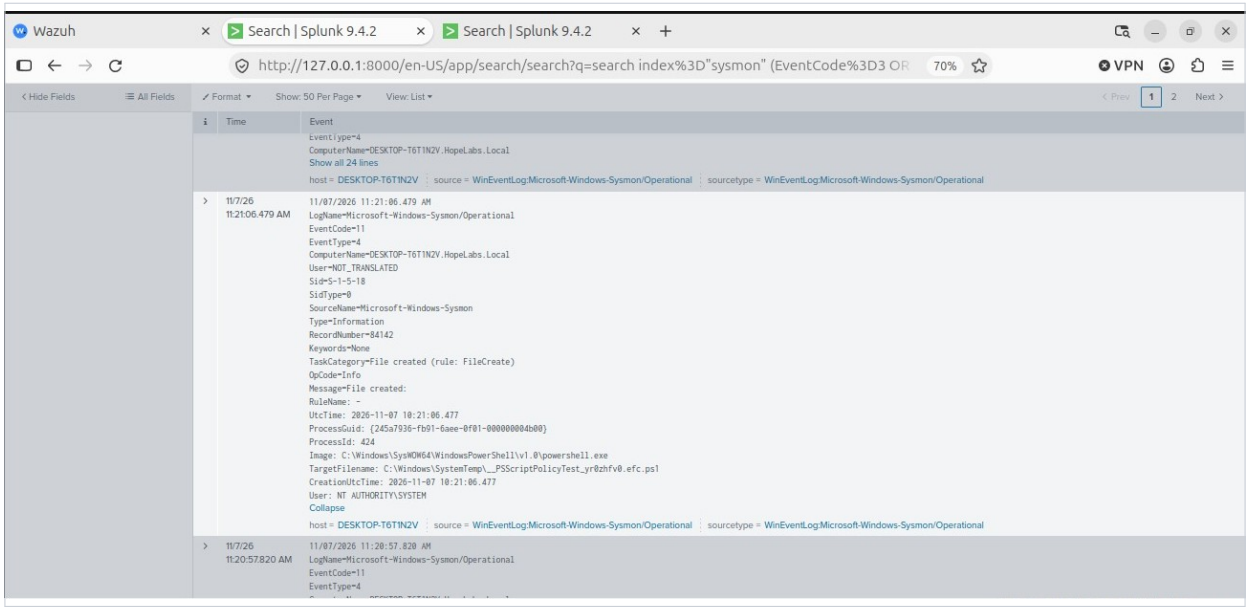


Figure 8 – Benign comparison: 32-bit PowerShell as SYSTEM writes the same kind of file to C:\Windows\SystemTemp (7 Nov, 11:21 AM)

Shows	Event 11: Image C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe, User NT AUTHORITY\SYSTEM, TargetFilename C:\Windows\SystemTemp__PSScriptPolicyTest_....ps1.
Demonstrates	The suspicious-looking file is created by PowerShell under different accounts and folders. The name alone proves nothing.
Limitation	Event is from 7 Nov, not the investigated window.

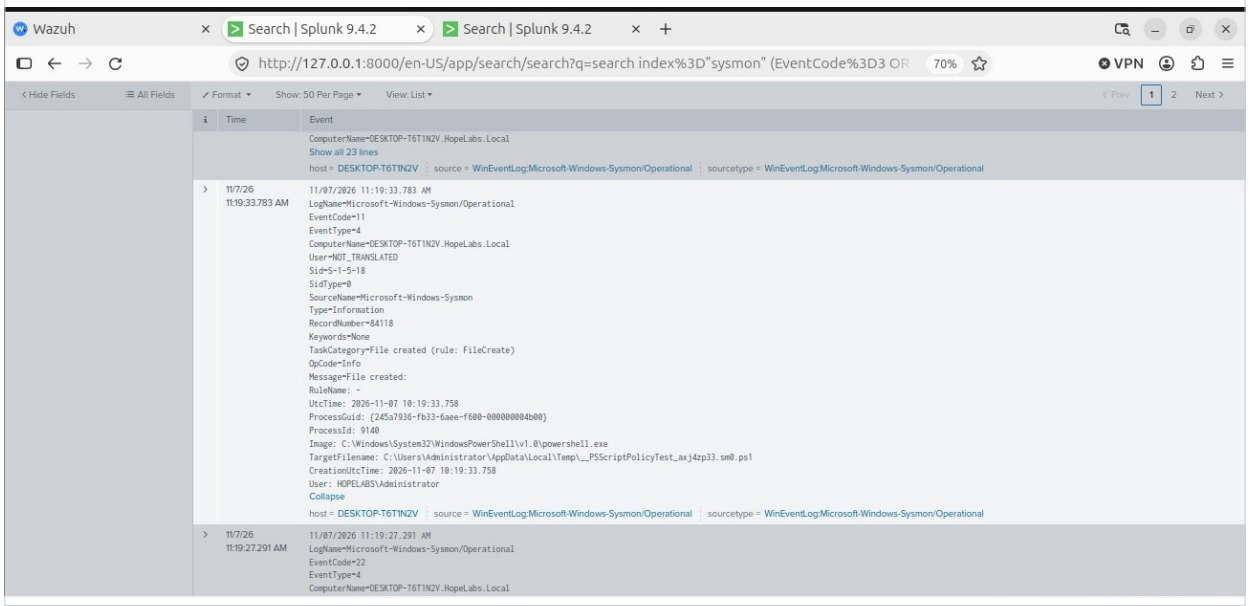


Figure 9 – Benign comparison: Administrator's PowerShell writes the same file to its Temp folder (7 Nov, 11:19 AM)

Shows	Event 11: HOPELABS\Administrator, powershell.exe PID 9140, TargetFilename ...\Temp__PSScriptPolicyTest_axj4zp33.sm0.ps1.
Demonstrates	Matches the files in the investigated chain: same user, same folder pattern.
Limitation	Another PowerShell session on 7 Nov (the author's own investigation).

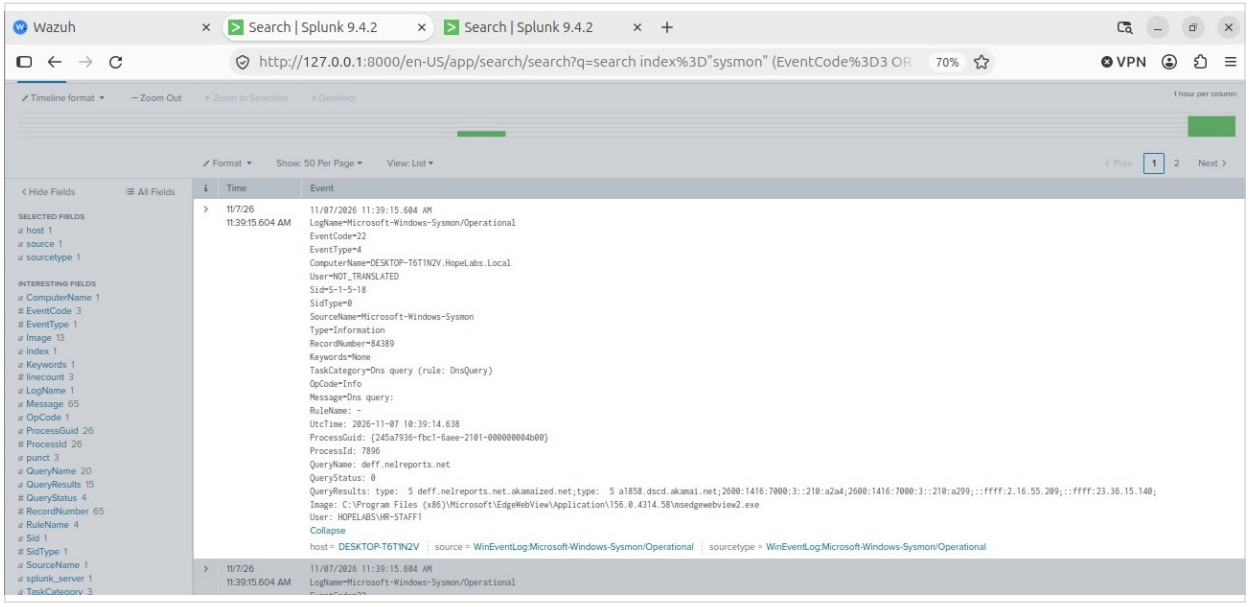


Figure 10 – Benign comparison: Edge WebView2 DNS lookup as HR-STAFF1 (7 Nov, 11:39 AM)

Shows	Event 22 from msedgewebview2.exe, user HOPELABS\HR-STAFF1, QueryName deff.nelreports.net.
Demonstrates	Real network activity looks like this: a named browser component and a known service. The investigated chain had none.
Limitation	Only one example of ordinary DNS traffic.

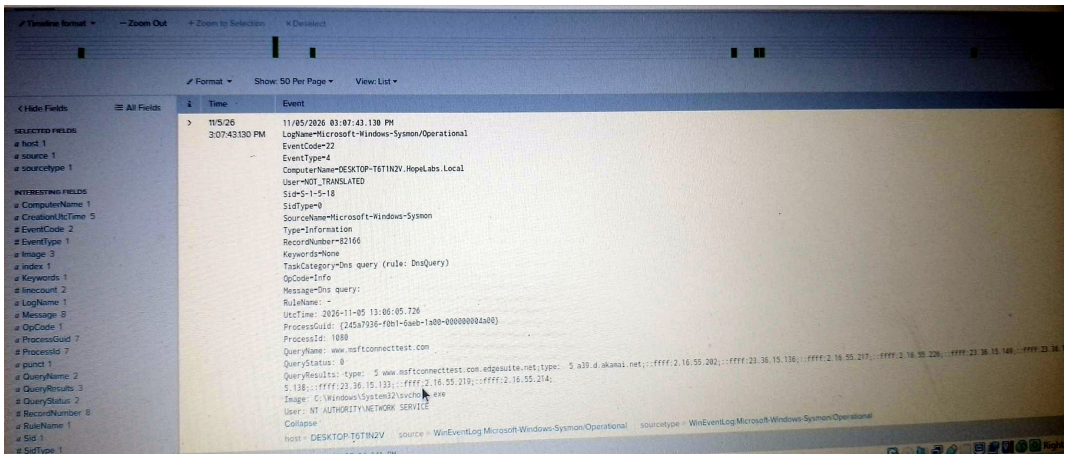


Figure 11 – Splunk event from inside the investigated window: svchost.exe checks internet connectivity (5 Nov, 3:07:43 PM; photo of screen)

Shows	Event 22 at 11/05/2026 3:07:43 PM, svchost.exe, NT AUTHORITY\NETWORK SERVICE, QueryName www.msftconnecttest.com.
Demonstrates	The only network-type event in the window was Windows' own connectivity check, not the PowerShell chain.
Limitation	Phone photo; the summary table of counts by process was not captured.

Step 6 – Rule out injection and Sysmon blind spots

Where	Windows 10, elevated PowerShell (7 Nov, about 1:14–1:19 PM)
Why	Event 8 (a process creating a thread inside another) is a classic injection signal. Event 255 means Sysmon lost events; "no network activity" is only trustworthy if Sysmon was keeping up.
Expected	Event 8 and 255 timestamps, none in the window.

Actually observed	No Event 8 or 255 between 3:00 and 3:10 PM on 5 Nov. Nearest drop: 12:58 PM on 5 Nov, over two hours earlier.

Windows 10, elevated PowerShell

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=8,255; StartTime=(Get-Date "2026-11-03")} |
 ForEach-Object { $m = $_.Message -replace "\r?\n"," "
  [pscustomobject]@{ Time=$_.TimeCreated; Id=$_.Id; Detail=$m.Substring(0,[Math]::Min(190,$m.Length)) } } |
 Format-Table -AutoSize -Wrap
```

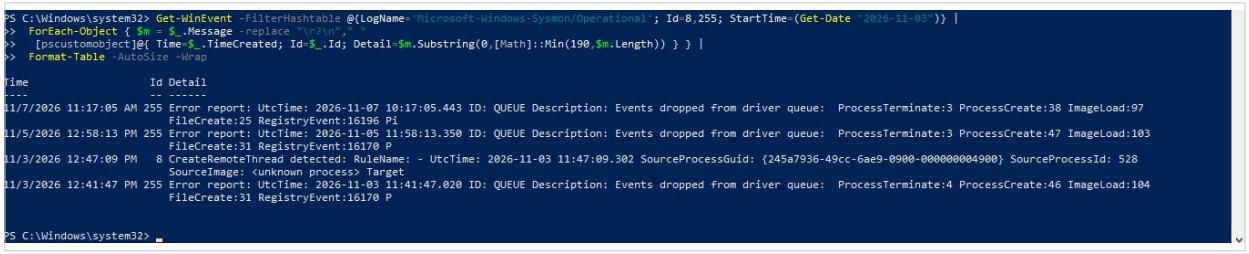


Figure 12 – Event 8 and 255 entries since 3 Nov: none between 3:00 and 3:10 PM on 5 Nov

Shows	Event 255 on 11/7 11:17 AM, 11/5 12:58 PM and 11/3 12:41 PM; Event 8 on 11/3 12:47 PM.
Demonstrates	No injection event and no Sysmon event-loss in the investigated window.
Limitation	Absence of an error message is strong, not absolute, proof that nothing was missed.

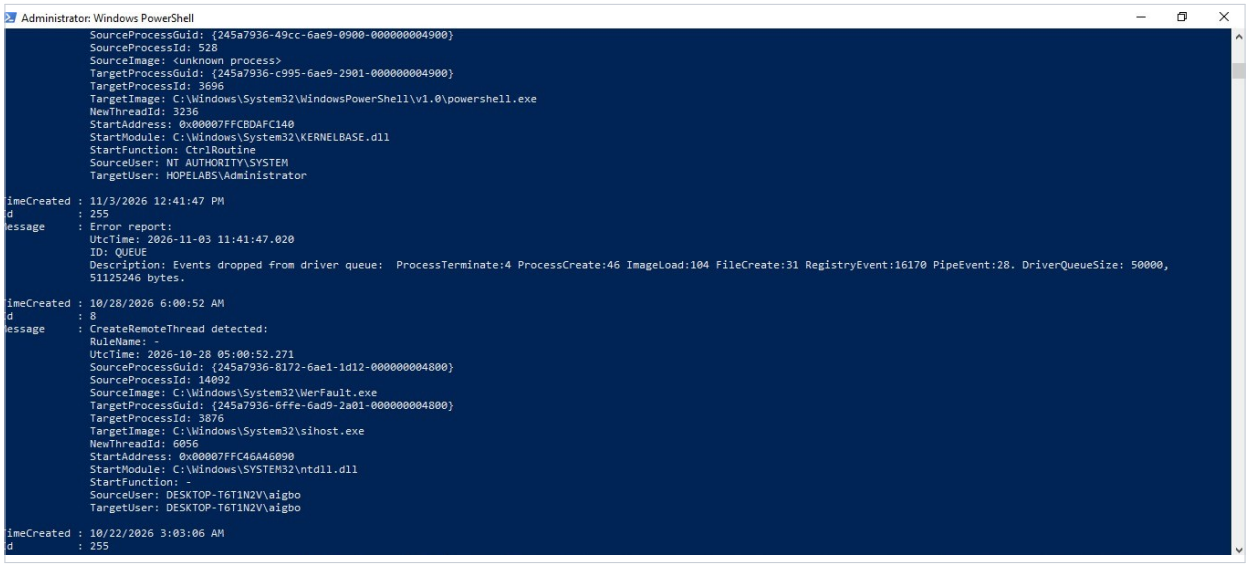


Figure 13 – Event 8 entries: WerFault.exe into sihost.exe (28 Oct) and an unknown process into PowerShell (3 Nov)

Shows	CreateRemoteThread from WerFault.exe (Windows Error Reporting) into sihost.exe, both as user aigbo; and an “<unknown process>” (SYSTEM) into powershell.exe PID 3696 with StartFunction CtrlRoutine. A Sysmon 255 entry shows 16,170 registry events dropped on 3 Nov.
Demonstrates	The Event 8 entries involve Microsoft components, same-user activity and the standard Ctrl+C routine. They look routine and are not in the investigated window.
Limitation	Routine-looking, not proven benign; PID 3696 is not part of the chain.

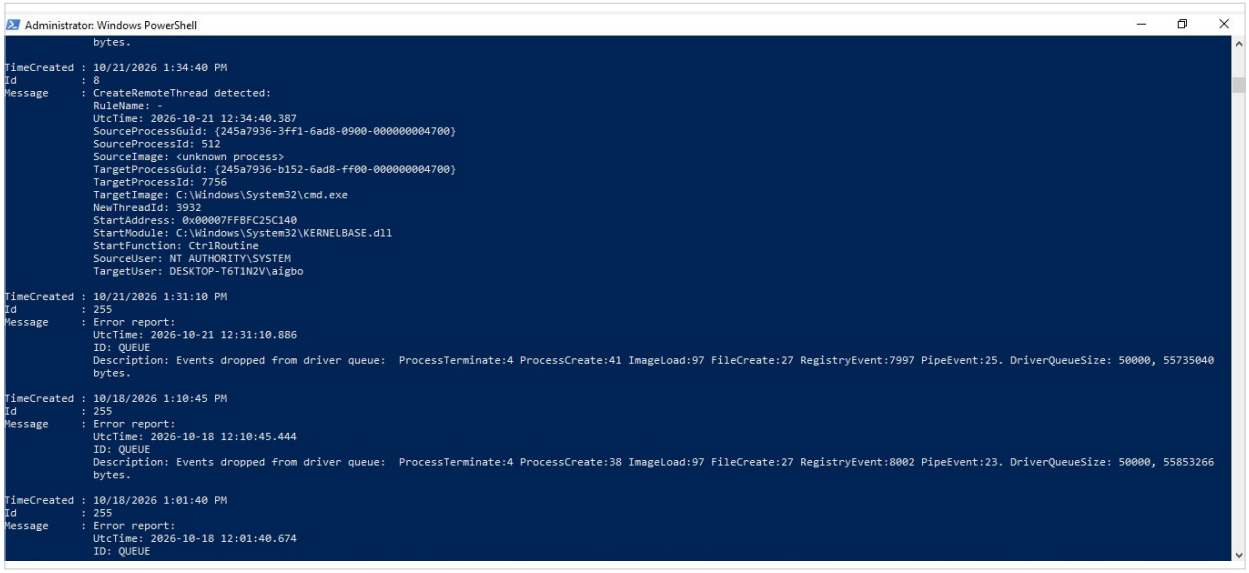


Figure 14 – Event 8 on 21 Oct: unknown process into cmd.exe with CtrlRoutine; more Event 255 entries

Shows	CreateRemoteThread into cmd.exe (PID 7756) with StartFunction CtrlRoutine, as user aigbo; Event 255 entries on 21 and 18 Oct.
Demonstrates	Same Ctrl+C pattern: the standard way Windows signals a console. Event 255 shows Sysmon can lose events during heavy activity.
Limitation	Historical entries, not the investigated window.

Step 7 – Turn the finding into a Wazuh detection

Where	Ubuntu VM (rule file, 7 Nov, about 1:28 PM) and Windows 10 (tests at about 1:40 and 1:45 PM)
Why	The chain's discovery commands (whoami, hostname) started by an encoded PowerShell are exactly what the tree showed. The earlier rules watched the PowerShell flags; this one watches what the encoded PowerShell goes on to do.
Expected	After adding the rule and restarting the manager, each run produces two alerts: one for whoami.exe and one for HOSTNAME.EXE.
Actually observed	Rule 100610 fired four times (two per run) at 13:40:19 and 13:45:03 on 7 Nov, level 12, with MITRE T1059.001, T1033 and T1082.

/var/ossec/etc/rules/local_rules.xml (appended after rules 100500 and 100600)

```
<group name="powershell,custom,">
  <rule id="100610" level="12">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)\\powershell\.exe$</field>
    <field name="win.eventdata.parentCommandLine" type="pcre2">(?i)\s-e(c|nc[a-z]*)?\s+[A-Za-z0-9+/=]{16,}</field>
    <field name="win.eventdata.image" type="pcre2">(?i)\\(whoami|hostname)\.exe$</field>
    <description>Encoded PowerShell spawned a discovery command (whoami/hostname)</description>
    <mitre>
      <id>T1059.001</id>
      <id>T1033</id>
      <id>T1082</id>
    </mitre>
  </rule>
</group>
```

Ubuntu VM terminal (syntax check, then restart)

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
```

EVIDENCE GAP

The syntax-check and restart output were not captured. That the rule loaded and fired is proven by the alerts themselves.

Windows 10, elevated PowerShell (harmless test; run as one line)

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -EncodedCommand dwBoAG8AYQBtAGkAOwAgAGgAbwBzAHQAbgBhAG0AZQA=
```

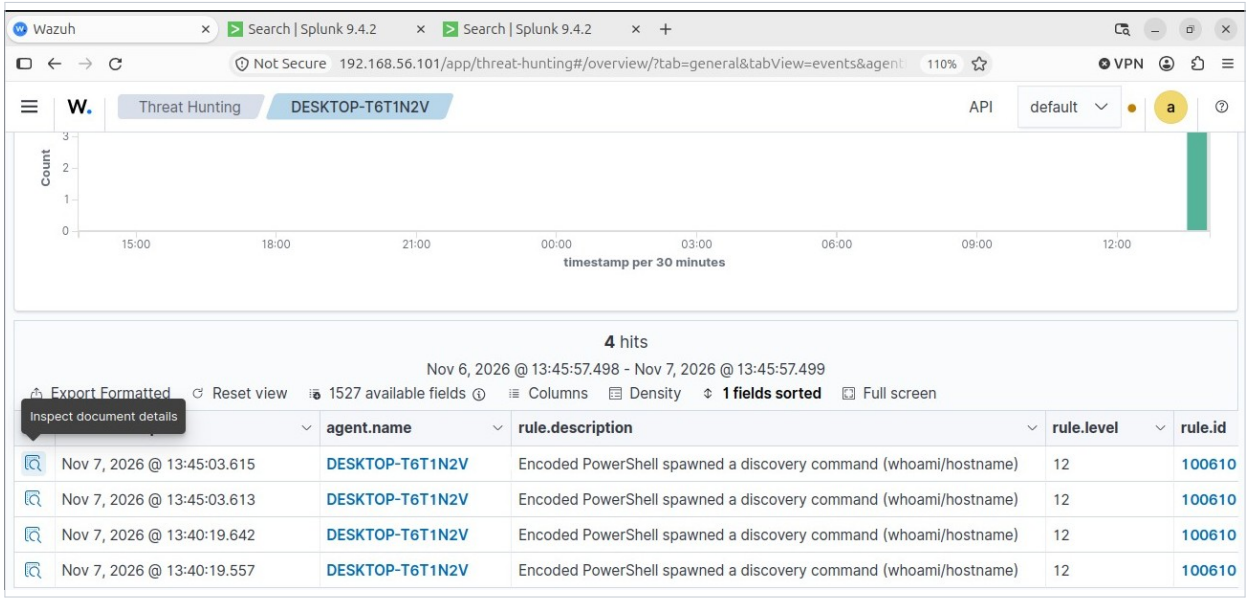


Figure 15 – Wazuh Threat Hunting: four alerts for rule 100610 (13:40:19 and 13:45:03 on 7 Nov)

Shows	Four hits for agent DESKTOP-T6T1N2V: 13:40:19.557, 13:40:19.642, 13:45:03.613 and 13:45:03.615; description “Encoded PowerShell spawned a discovery command (whoami/hostname)”; level 12; rule 100610.
Demonstrates	The new rule works: each run of the encoded test produced one alert per discovery command, so the whole chain is now detected.
Limitation	Both runs were the author’s own tests, so this shows detection, not false-positive behaviour.

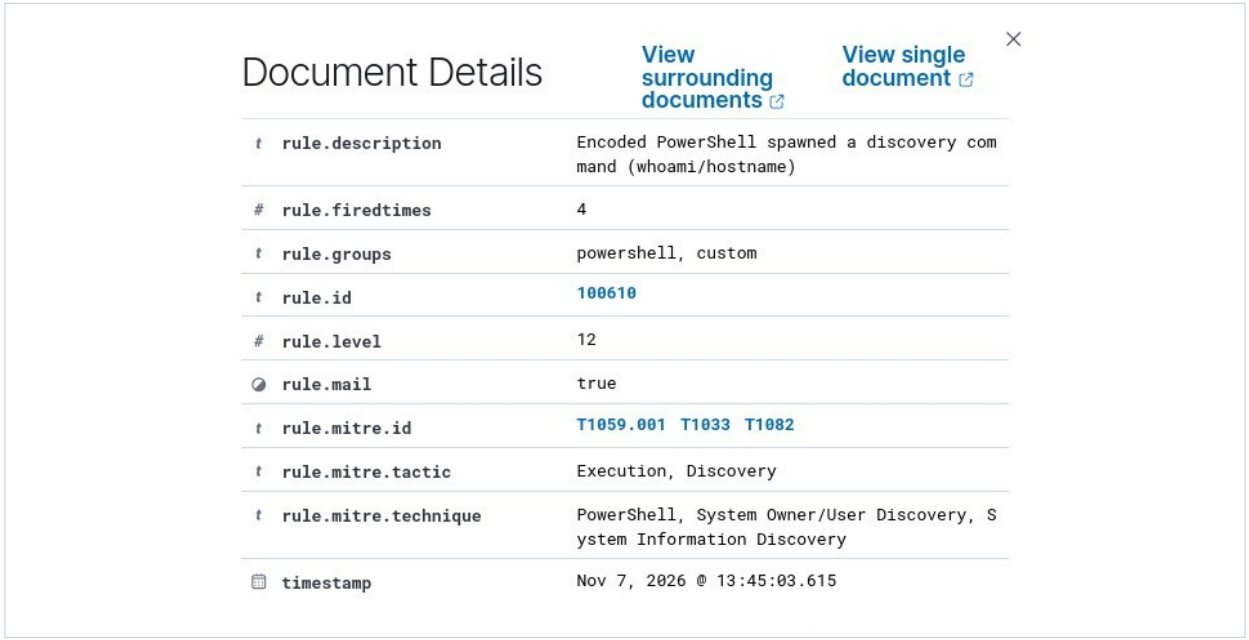


Figure 16 – Rule 100610 alert metadata: level 12 and MITRE T1059.001, T1033, T1082

Shows	rule.id 100610; level 12; groups powershell, custom; mitre.id T1059.001, T1033, T1082; tactics Execution, Discovery; techniques PowerShell, System Owner/User Discovery, System Information Discovery; firedtimes 4; mail true.
Demonstrates	The MITRE mapping covers both the execution (PowerShell) and the discovery behaviour.
Limitation	None.

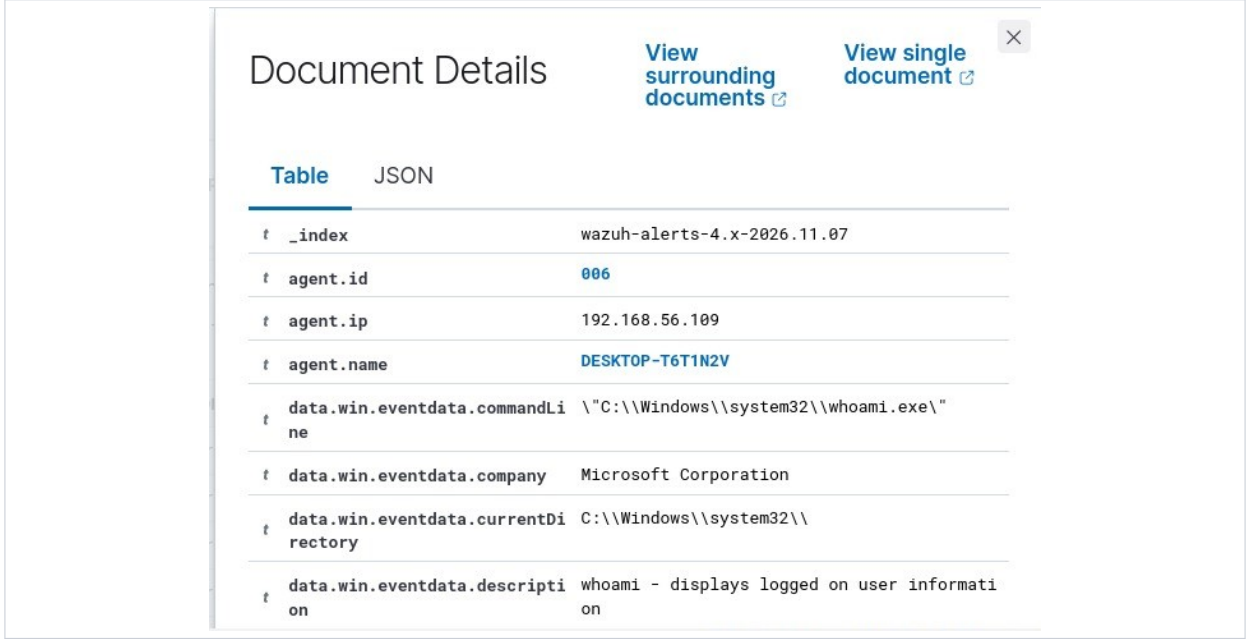


Figure 17 – Alert for whoami.exe: the discovery command seen by Wazuh

Shows	commandLine “C:\Windows\system32\whoami.exe” on agent 006; description “whoami - displays logged on user information”.
Demonstrates	The child process in the alert is the whoami.exe from the process tree.
Limitation	None.

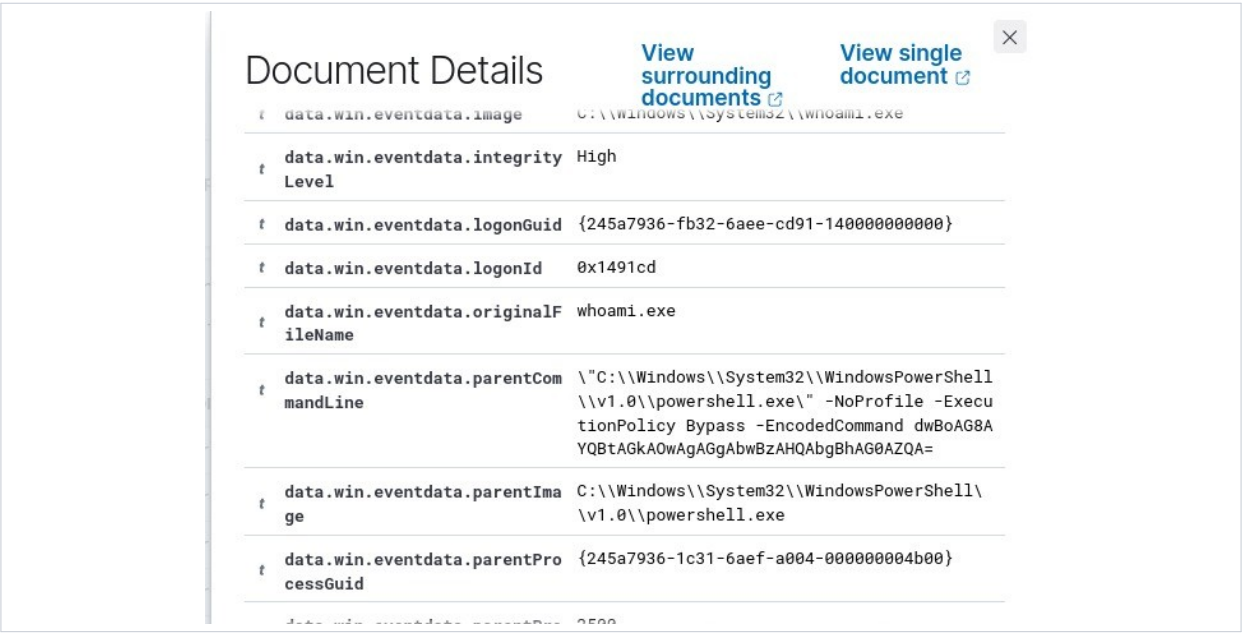


Figure 18 – Alert for whoami.exe: its parent is the encoded PowerShell

Shows	parentCommandLine "...powershell.exe -NoProfile -ExecutionPolicy Bypass -EncodedCommand dwBoAG8A..."; parentImage powershell.exe; parentProcessGuid {245a7936-1c31-6aef-a004-000000004b00}; parentProcessId 2500.
Demonstrates	The alert carries the parent's encoded command line, which is the condition rule 100610 requires.
Limitation	None.

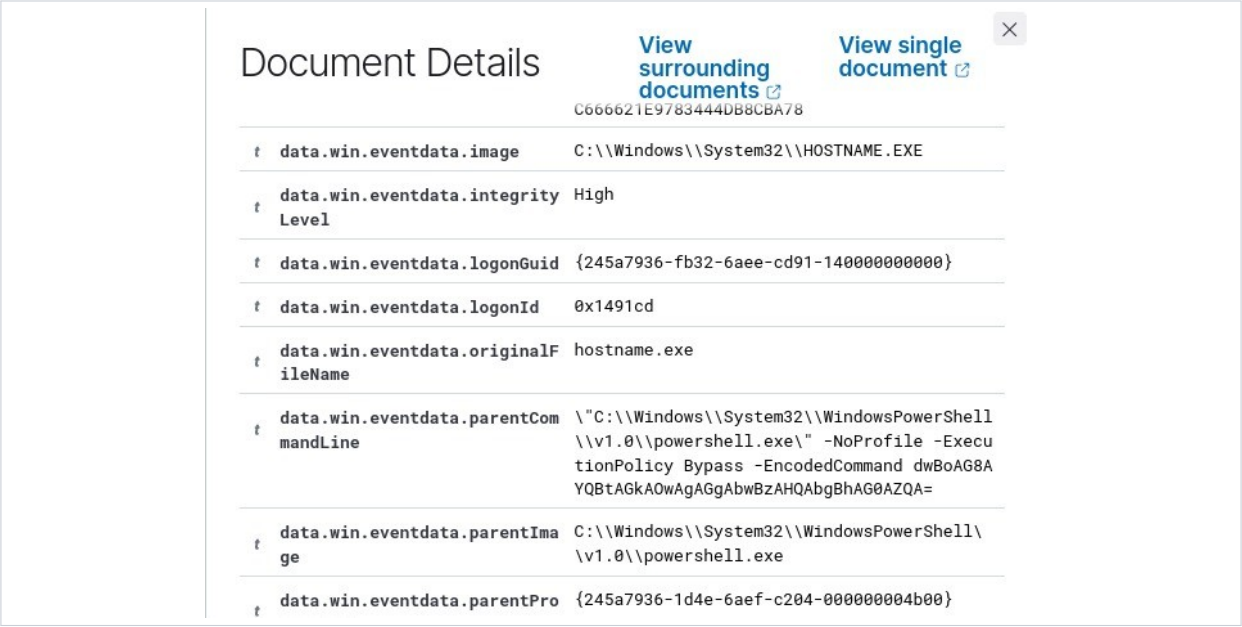


Figure 19 – Alert for HOSTNAME.EXE: the same encoded parent (second run)

Shows	parentCommandLine with -EncodedCommand dwBoAG8A...; parentImage powershell.exe; parentProcessGuid {245a7936-1d4e-6aef-c204-000000004b00}.
Demonstrates	The second run produced the same pattern under a new parent process (different GUID), as expected for a fresh test.
Limitation	The Wazuh timestamp is about one second earlier than the Sysmon event time, so the VM clocks are slightly out of step; clock sync was not checked.

Step 8 – Record the conclusion

Where	Written analysis (Chapter 8)
Why	The investigation ends with a decision and the reasons for it.
Expected	A clear verdict with evidence for and against.
Actually observed	Verdict: suspicious technique, benign intent (authorised lab test). See Section 8.5.

Chapter 7: Investigation Logic and Why Context Matters

IN PLAIN ENGLISH
We scored the chain on clues that point toward trouble and clues that point away. A scary-looking flag alone does not decide the case.

Table 7 – Indicators weighed in the investigation

Indicator	Points toward	Evidence	Weight
-EncodedCommand	Suspicious	Fig. 3	High (common attacker technique)
-ExecutionPolicy Bypass	Suspicious	Fig. 3	Medium
-WindowStyle Hidden	Suspicious	Fig. 3	Medium
Administrator elevation from a standard user's desktop session	Notable	Fig. 5, Fig. 6	Medium: a real SOC would ask who authorised it
Children whoami.exe and HOSTNAME.EXE	Mildly suspicious	Fig. 3	Low: reconnaissance-style commands, but harmless alone
No network or DNS activity from the chain	Benign	Fig. 7	High
Only PSScriptPolicyTest files created	Benign	Fig. 7, Fig. 8	High
No process-injection events in the window	Benign	Fig. 12	Medium
No Sysmon gaps in the window	Supports reliability	Fig. 12	Medium
Parent is an interactive console; short, isolated bursts	Benign	Fig. 4	Medium
Persistence (registry, scheduled tasks) checked	Unknown	Not examined	Gap

7.1 True positive, benign intent

The detections in the previous lab were **true positives for the technique**: the behaviour really matched an attacker pattern. The investigation here decides intent. The deciding context is that the lab owner generated the activity, the chain touched nothing outside the machine, and the only files were routine PowerShell self-tests.

7.2 What would change the verdict

- A network connection or DNS lookup to an unknown address from any chain process.
- A file written to Temp, Startup or a user folder that is not the PSScriptPolicyTest pattern.
- An Event 8 or an Event 255 inside the window.
- PowerShell started by Word, Excel, a browser or a script host instead of a console.
- The same command line on other machines, or by other accounts.
- No one able to say who authorised the Administrator session.

7.3 The new detection: rule 100610

Table 8 – Rule 100610 line by line

Line	Plain meaning
rule id 100610, level 12	Our own ID (custom rules start at 100000) and a high severity, equal to the built-in encoded-command alert
if_sid 61603	Only look at events Wazuh already classifies as Sysmon Event 1 (process created)
parentImage ... powershell.exe$	The parent program must be PowerShell (case ignored; the path must end there)
parentCommandLine: -e / -ec / -enc... plus 16 or more base64 characters	The parent was started with an encoded command (long or short flag form)
image ... whoami.exe or hostname.exe	The child that was started must be one of these discovery commands
mitre T1059.001, T1033, T1082	PowerShell; System Owner/User Discovery (whoami); System Information Discovery (hostname)

All three conditions must be true together, so ordinary use of whoami or hostname does not match. The rule would also fire for a legitimate administrator who runs these commands from an encoded script; that false-positive risk was not measured.

Chapter 8: Results and Conclusion

8.1 Test results

Table 9 – Checks, results and status

#	Check	Result	Status
C1	Sysmon records events 1, 3, 11, 22	All present	Verified
C2	Process tree reconstructed	Two consoles, four PowerShell children, four whoami/HOSTNAME children	Verified
C3	Ancestor of the consoles	RuntimeBroker PID 6800 (HR-STAFF1 session)	Verified (owner from separate event)
C4	Network or DNS from chain	None	Verified
C5	Files from chain	Five PSScriptPolicyTest files only	Verified
C6	Injection (Event 8) in window	None	Verified
C7	Sysmon event loss (255) in window	None; nearest 12:58 PM	Verified
C8	Splunk comparison activity	Benign DNS and file events	Verified
C9	Persistence (registry/scheduled tasks)	Not examined	Not tested
C10	consent.exe linked to elevation	Plausible, unproven	Unverified
C11	Rule 100610 fires on the encoded test	Four alerts (whoami and HOSTNAME, two runs)	Verified
C12	Rule 100610 false positives	No benign encoded scripts run	Not tested
C13	Rule syntax-check and restart output	Not captured	Evidence gap

8.2 Timeline

Table 10 – Key times (5 Nov, local)

Time	Event
12:58	Sysmon drops events (Event 255) – before the test
13:01:38	PowerShell console PID 7672 started by RuntimeBroker (HR-STAFF1 session), as Administrator
15:05:40	Run 1: encoded PowerShell 6724; whoami and HOSTNAME children at 15:05:41; hidden PowerShell 8876 at 15:05:41
15:06:13	consent.exe (permission prompt)
15:06:58	PowerShell console PID 7240 started (parent 6800)
15:07:03	Run 2: encoded PowerShell 9492; children at 15:07:04; hidden PowerShell 2444 at 15:07:04
15:07:43	svchost connectivity check (only network event in the window)

8.3 Investigation notes

Table 11 – Investigation notes

Item	Note
Case	Lab investigation of the PowerShell chain from the PowerShell Detection Lab
Host / user	DESKTOP-T6T1N2V (192.168.56.109); HOPELABS\Administrator, High integrity; parent session HOPELABS\HR-STAFF1

Item	Note
Trigger	Encoded + bypass + hidden PowerShell (see the earlier Wazuh alerts 92057, 91815 and custom rule 100600)
Process chain	RuntimeBroker 6800 → powershell 7672 / 7240 → encoded and hidden PowerShell → whoami / HOSTNAME (Table 5)
Command-line context	Encoded payload decodes to “whoami; hostname”; hidden command is Get-Process | Out-Null
Network / file	No network or DNS; five PSScriptPolicyTest files only
Reliability checks	No Event 8 or 255 in the window
Open items	Registry and scheduled-task review; who authorised the elevation

8.4 The six analyst questions

Table 12 – Answers tied to evidence

Question	Answer
What happened?	Two administrator PowerShell consoles, opened from HR-STAFF1’s desktop session, each ran an encoded command (decoded: whoami; hostname) and a hidden-window process listing, on 5 Nov at 3:05 and 3:07 PM.
How was it detected?	Sysmon Event 1 recorded each process; the earlier Wazuh alerts (92057, 91815, 100600) flagged the behaviour; this lab reconstructed the tree from Sysmon.
What evidence supports the finding?	Process table with PIDs and parents (Fig. 3), ancestors (Fig. 5), no network/file payload (Fig. 7), no injection or Sysmon gaps (Fig. 12).
Which host and account?	DESKTOP-T6T1N2V; HOPELABS\Administrator (High integrity); desktop session of HOPELABS\HR-STAFF1.
What else should be checked?	Registry changes (Event 13) and scheduled tasks around the window; the same command line on other hosts; other logons by the account; who approved the Administrator elevation; Event 3 from these processes in Wazuh.
What action in a real SOC?	Confirm with the account owner and change records whether the activity was authorised. If not: isolate the host, preserve logs, reset the Administrator credentials, hunt the command line fleet-wide, and document. Here: authorised test, document and close.

8.5 Analyst conclusion

> **VERIFIED RESULT**
> **Verdict: suspicious technique, benign intent.** The chain uses three recognised attacker techniques (encoded command, execution-policy bypass, hidden window) and a recon-style pair of commands. In context, however, it was started from a known console, produced no network or DNS activity, wrote only routine PowerShell self-test files, involved no injection, and occurred while Sysmon was healthy. It is closed as an **authorised lab test (true-positive behaviour, not malicious)**, with two open items: review registry and scheduled-task activity, and confirm authorisation of the elevation. Context decided the outcome, not the flags.

Chapter 9: Evidence and Screenshot Analysis

Each figure was introduced where it supports a step in Chapter 6, with a table of what it shows, demonstrates and cannot prove. This chapter lists the evidence by deliverable.

Table 13 – Deliverables and their evidence

Deliverable	Evidence
Process tree	Fig. 4 (drawn) and Fig. 3 (raw Sysmon output)
Investigation notes	Section 8.3
Evidence screenshots	Fig. 2, Fig. 3, Fig. 5, Fig. 6, Fig. 7, Fig. 8, Fig. 9, Fig. 10, Fig. 11, Fig. 12, Fig. 13, Fig. 14
Wazuh detection	Fig. 15, Fig. 16, Fig. 17, Fig. 18, Fig. 19
Analyst conclusion	Section 8.5

Chapter 10: Troubleshooting and Root-Cause Analysis

Table 14 – Issues, causes and fixes

#	Symptom	Cause	Fix	Confirmed?
1	Splunk results showed events from 7 Nov, not 5 Nov	The time picker (“Last 24 hours”) no longer reached 5 Nov; the query’s own earliest/latest was not applied as intended. Only the picker was verified	Re-ran with an explicit window; the 12:02 photo then shows 3:07:43 PM events from 5 Nov (Fig. 11)	Probable (picker confirmed; exact query not verified)
2	Splunk summary table of counts by process not captured	Screenshots taken of raw events instead	Used individual events as the comparison; noted as a gap	Yes
3	Concern that Sysmon might have missed events	Event 255 entries exist elsewhere in the log	Checked the window: none in 3:00–3:10 PM (Fig. 12)	Yes
4	Duplicate screenshot uploads	Same Step 3 image sent twice	Resent the Step 4 command; no impact on evidence	Yes
5	Encoded test printed PowerShell help and “The value specified with -EncodedCommand is not properly encoded”	Pasted lines merged into encenc = ... inside the command, so the value was not valid Base64; whoami and hostname never started	Ran a single line with the literal Base64 string; the alerts then fired (Fig. 15)	Yes (visible in the screenshot)

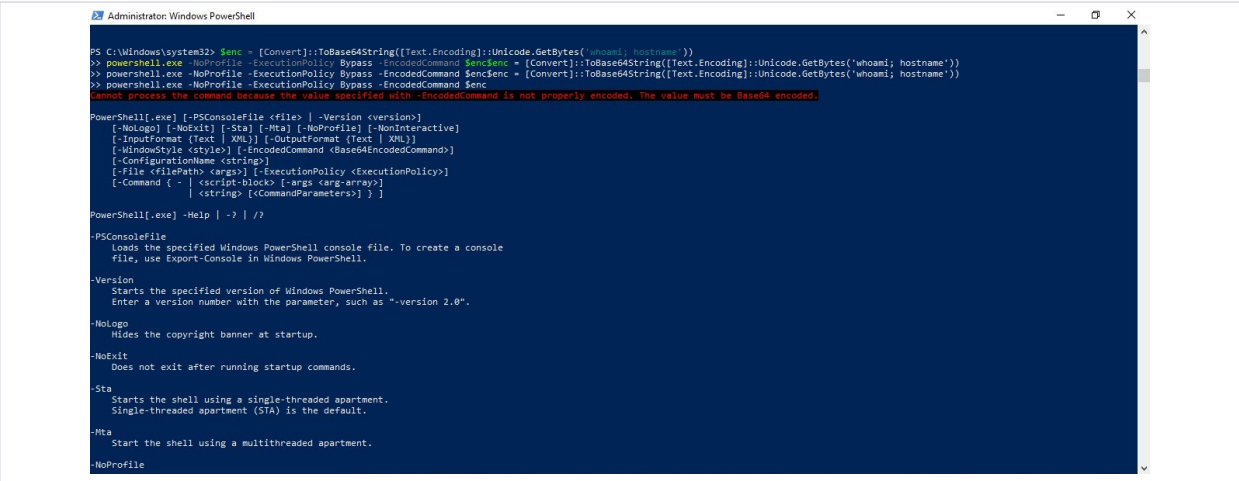


Figure 20 – The failed test: pasted lines merged into encenc, giving “not properly encoded” (7 Nov, 1:40 PM)

Shows	A garbled command line containing encenc = [Convert]::ToBase64String(...), the red error “Cannot process the command because the value specified with -EncodedCommand is not properly encoded”, then PowerShell’s help text.
Demonstrates	A detection can only fire if the activity really happens: nothing ran, so nothing could alert.
Limitation	Left in the report on purpose; this was a typing problem, not a rule problem.

> **NOTE**
> Lesson: when you investigate days later, “last 24 hours” silently moves. Always state the exact start and end time in the query and in your notes.

Chapter 11: Security Findings and Risk Analysis

Table 15 – Findings

ID	Finding	Evidence	Confidence
F1	Encoded + bypass + hidden PowerShell launched in two bursts by an Administrator console	Fig. 3	Confirmed
F2	Standard desktop user's session spawned elevated PowerShell consoles	Fig. 5, Fig. 6	Confirmed (owner inferred from PID 6800)
F3	No network, DNS or payload activity from the chain	Fig. 7	Confirmed for the 3-minute window
F4	Sysmon drops events under load (Event 255)	Fig. 12, Fig. 13	Confirmed; impact depends on timing
F5	Event 8 entries look routine	Fig. 13, Fig. 14	Probable, not proven

CAUTION

F2 is the finding a real SOC would escalate even if the commands were harmless: an HR user's session producing administrator PowerShell needs an owner and a reason.

Chapter 12: Mitigation and Remediation

Table 16 – Implemented versus recommended

Measure	Status	Benefit	Verification
Investigate with Sysmon process tree and context	Done	Evidence-based verdict	Chapters 6–8
Record network, file and DNS events in Sysmon	Already in place	Makes absence meaningful	Fig. 2
Wazuh rule 100610: encoded PowerShell spawning whoami/hostname	Implemented	Detects what the chain does, not just how it starts	Fig. 15
Review registry (Event 13) and scheduled tasks for the window	Recommended	Closes the persistence gap	Event 13 / Event 4698 results
Reduce Sysmon event loss (larger queue, filter noisy events)	Recommended	Fewer blind spots	Fewer Event 255 entries
Alert when PowerShell is started from a standard user's session with elevation	Recommended	Catches F2	Test elevation triggers alert
Limit who can run Administrator PowerShell; use just-enough-administration	Recommended	Reduces F2	Standard user cannot elevate

Chapter 13: What I Learned

> **NOTE**
> Drafted from the actual investigation; edit the wording into your own voice.

Concepts. A process tree is a family tree: who started whom. The command line says what was asked; the children say what actually happened; network and file events say what it touched. Together they tell a story that one scary-looking flag cannot.

Practical skills. I used PowerShell to turn Sysmon events into a table of PID, parent PID, user and command line; traced the parents to find the top of the chain; searched for network, DNS and file events from specific PIDs; and compared with ordinary activity in Splunk.

Context. The __PSScriptPolicyTest file looked suspicious but is normal. Empty network results were a valid finding, but only after I checked Sysmon was not dropping events. I also learned that my own investigation commands create more evidence.

Detection engineering. I turned what the process tree showed into a Wazuh rule that needs three things at once (encoded PowerShell parent, whoami or hostname child, Sysmon process event), and watched it fire four times. I also learned that a failed test (a garbled paste) shows nothing, so I must always confirm the activity actually ran.

Dates. “Last 24 hours” moved a day and hid the data I needed. Stating the exact window is part of good evidence handling.

Still to develop. Persistence checks (registry and scheduled tasks), tuning Sysmon, and escalation decisions around elevation from standard user sessions.

Chapter 14: Practical Knowledge and Competencies

Table 17 – Skills mapped to evidence

Skill	Evidence	Level	Next exercise
Reconstruct a process tree from Sysmon Event 1	Fig. 3, Fig. 4	Demonstrated hands-on	Do it in Splunk
Trace parent processes	Fig. 5	Demonstrated hands-on	Trace a Word-to-PowerShell chain
Search network/file/DNS by PID	Fig. 7	Demonstrated hands-on	Add Event 13 registry search
Check telemetry reliability (Event 255, Event 8)	Fig. 12	Demonstrated hands-on	Tune Sysmon queue
Weigh indicators and reach a verdict	Chapter 7	Demonstrated hands-on	Write an escalation note
Write a process-chain Wazuh rule (parent and child fields)	Fig. 15, Fig. 16	Demonstrated hands-on	Measure false positives and tune
Persistence hunting	—	Studied only	Check Run keys and scheduled tasks

Chapter 15: Reproduction Guide (for a beginner)

> **IN PLAIN ENGLISH**
> You need a Windows computer with Sysmon and a past event to investigate. If you did the PowerShell Detection Lab, use its 5 Nov test window, otherwise create a fresh one (Section 15.2). Replace the dates with yours.

15.1 What you need

- Windows 10 VM with Sysmon recording process, network and file events; Administrator PowerShell.
- Optional: Splunk with the Sysmon data indexed (for comparison).
- The exact start and end time of the activity (check the time zone).

15.2 Create something to investigate (only if you have no earlier test)

Windows 10, elevated PowerShell (harmless test)

```
$cmd = 'whoami; hostname'
$enc = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($cmd))
powershell.exe -NoProfile -ExecutionPolicy Bypass -EncodedCommand $enc
powershell.exe -NoProfile -WindowStyle Hidden -Command "Get-Process | Out-Null"
```

15.3 Run the six checks

1. **Count Sysmon events** (Step 1 command). You should see Event 1, 3, 11 and 22.
2. **Build the process tree** (Step 2 command; change the dates). You should see your PowerShell processes with PID and PPID. Draw the tree by joining each PPID to its PID.
3. **Trace the ancestors** (Step 3 command; use the PPIDs of your top processes). You should see what started your consoles.
4. **Search network, file and DNS events** (Step 4 command; put your PIDs in $p). Empty output is valid.
5. **Check Event 8 and 255** (Step 6 command). You should see their times; note any in your window.
6. **Compare with normal activity in Splunk** (Step 5 query; state the exact window).

15.3b Optional: turn it into a Wazuh detection

1. Add rule 100610 (Appendix D) to the end of /var/ossec/etc/rules/local_rules.xml once only. Check the ID is free first: sudo grep -rn "100610" /var/ossec/etc/rules/.
2. Check the syntax with sudo /var/ossec/bin/wazuh-analysisd -t (no errors), then sudo systemctl restart wazuh-manager.
3. On Windows, run the one-line encoded test from Step 7.
4. In the dashboard search rule.id: 100610 (last 15 minutes). You should see two alerts per run.

15.4 Write the conclusion

List indicators pointing toward and away from malicious intent (Table in Chapter 7), state your verdict, and list what would change it. Answer the six analyst questions (Section 8.4).

15.5 Troubleshooting

Table 18 – Quick guide

Symptom	Cause	Fix
Empty tree table	Wrong dates or time zone	Use local times; widen the window
Splunk shows wrong dates	“Last 24 hours” moved	Put earliest= and latest= in the query
Cannot find the parent	Parent started before the window	Widen the start time; PIDs get reused, so check the time
No Event 3/11/22 at all	Sysmon config not logging them	Update the Sysmon configuration

Chapter 16: Limitations and Future Improvements

- **Controlled test, one host.** The activity was generated by the author, so “benign” is known, not discovered.
- **Persistence not checked** (registry, scheduled tasks).
- **Splunk count-by-process table not captured.**
- **Elevation link inferred:** RuntimeBroker PID 6800’s owner comes from a separate event; consent.exe link unproven.
- **Event 8 entries** judged routine from their pattern, not proven.
- **Rule 100610 untuned:** its false-positive rate was not measured and the syntax-check output was not captured.
- **Clock difference:** Wazuh timestamps are about one second earlier than the Sysmon event time; clock sync was not checked.
- **Some evidence is a phone photo** of the monitor (Fig. 11).

Future improvements

- Add Event 13 and scheduled-task review; build a saved Splunk search for process trees; tune Sysmon to reduce event loss; create an escalation template for “elevation from a standard user session”.

Chapter 17: Conclusion

The investigation reconstructed the PowerShell chain from Sysmon, traced it to a desktop session, found no network, DNS or payload activity, ruled out injection and Sysmon blind spots in the window, and concluded: **suspicious technique, benign intent**. It then turned the finding into a new Wazuh rule (100610) that fired on the same chain. The key skill demonstrated is judging behaviour in context rather than by its flags.

Next steps: close the persistence gap, document the authorisation of elevated PowerShell, and continue the program.

Appendix A: Evidence Register

Originals are unchanged; figures are cropped copies. Phone photos had mild contrast and sharpness adjustment. No content was edited, redrawn or removed.

Table 19 – Figure-to-original evidence register

Fig.	Description	Original file(s)
1	The investigation brief (source of tasks, deliverables and analyst questions)	1000496689.jpg
2	Sysmon event counts by type (latest 5,000 events)	1000496997.jpg
3	Every process created between 3:05 and 3:08 PM on 5 Nov, with PID, parent PID, user and command line	1000497000.jpg
4	Process tree	Drawn for this report from Sysmon Event 1 data (not a screenshot)
5	PID 7672 was started by RuntimeBroker.exe (PID 6800) at 13:01:38	1000497012.jpg
6	From the earlier lab: the same RuntimeBroker PID 6800 shown with ParentUser HOPELABS\HR-STAFF1 (starting PowerShell PID 4568)	1000494664.jpg
7	Network, DNS and file events from the chain's processes: only five __PSScriptPolicyTest files	1000497018.jpg
8	Benign comparison: 32-bit PowerShell as SYSTEM writes the same kind of file to C:\Windows\SystemTemp (7 Nov, 11:21 AM)	1000497043.jpg (also 1000497045.jpg, 1000497046.jpg)
9	Benign comparison: Administrator's PowerShell writes the same file to its Temp folder (7 Nov, 11:19 AM)	1000497044.jpg
10	Benign comparison: Edge WebView2 DNS lookup as HR-STAFF1 (7 Nov, 11:39 AM)	1000497042.jpg
11	Splunk event from inside the investigated window: svchost.exe checks internet connectivity (5 Nov, 3:07:43 PM; photo of screen)	20261107_120228797994476726828904 8.jpg
12	Event 8 and 255 entries since 3 Nov: none between 3:00 and 3:10 PM on 5 Nov	1000497118.jpg
13	Event 8 entries: WerFault.exe into sihost.exe (28 Oct) and an unknown process into PowerShell (3 Nov)	1000497111.jpg (also 1000497112.jpg, 1000497113.jpg)
14	Event 8 on 21 Oct: unknown process into cmd.exe with CtrlRoutine; more Event 255 entries	1000497114.jpg (also 1000497115.jpg)
15	Wazuh Threat Hunting: four alerts for rule 100610 (13:40:19 and 13:45:03 on 7 Nov)	1000497161.jpg (also 1000497164.jpg)
16	Rule 100610 alert metadata: level 12 and MITRE T1059.001, T1033, T1082	1000497189.jpg (also 1000497204.jpg, 1000497201.jpg)
17	Alert for whoami.exe: the discovery command seen by Wazuh	1000497192.jpg (also 1000497195.jpg)
18	Alert for whoami.exe: its parent is the encoded PowerShell	1000497207.jpg
19	Alert for HOSTNAME.EXE: the same encoded parent (second run)	1000497171.jpg (also 1000497165.jpg, 1000497168.jpg)
20	The failed test: pasted lines merged into encenc, giving "not properly encoded" (7 Nov, 1:40 PM)	1000497141.jpg (also 1000497142.jpg, 1000497143.jpg)

Appendix B: Commands Used

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5000 | Group-Object Id | Sort-Object Count -Descending | Format-Table Count, Name -AutoSize
# Steps 2, 3, 4 and 6: see Chapter 6 (full commands)
```

Appendix C: Splunk Query

```
index=sysmon (EventCode=3 OR EventCode=11 OR EventCode=22)
  earliest="11/05/2026:15:05:00" latest="11/05/2026:15:08:00"
  | stats count by EventCode, Image, User
```

Appendix D: Wazuh Rule 100610 (copy-ready)

Append to /var/ossec/etc/rules/local_rules.xml (once only)

```
<group name="powershell,custom,">
  <rule id="100610" level="12">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)\\powershell\.exe$</field>
    <field name="win.eventdata.parentCommandLine" type="pcre2">(?i)\s-e(c|nc[a-z]*)?\s+[A-Za-z0-9+/=]{16,}</field>
    <field name="win.eventdata.image" type="pcre2">(?i)\\(whoami|hostname)\.exe$</field>
    <description>Encoded PowerShell spawned a discovery command (whoami/hostname)</description>
    <mitre>
      <id>T1059.001</id>
      <id>T1033</id>
      <id>T1082</id>
    </mitre>
  </rule>
</group>
```

Appendix E: Glossary in Plain Language

Table 20 – Glossary

Term	Plain meaning
Child process	A program started by another program
Command line	The text used to start a program
Context	The surrounding facts that give an event its meaning
Elevation	Running a program with administrator rights
Injection	One program forcing its code into another program
Persistence	A way an attacker keeps access after a restart
True positive	An alert that correctly detected the behaviour

Appendix F: Integrity and Redaction Note

- Originals preserved; cropped copies used. No values, errors, timestamps or IDs altered.
- No redaction applied. Visible: lab IPs, host and user names (including local user aigbo in Event 8 entries). Consider redacting before publishing publicly.
- The process-tree figure was drawn from the Sysmon table; it is not a screenshot.